

HACKFEST 2026

Problem Statement Documentation & Technical Submission

SECTION 1 — TEAM DETAILS

Team ID	KH110
Team Name	Trojan Warriors
Team Leader Name	Soham Parab
Team Members	Vaibhav Kawa, Channveer Mulge, Yugandhar Shende
Problem Statement No.	PS12
Problem Statement Name	Agentic Tutor Orchestrating Subject-Specialist Sub-Agents
Date of Submission	11-09-2026

SECTION 2 — UNDERSTANDING OF THE PROBLEM

Problem Statement

Conventional AI tutoring systems lack subject specialization and struggle to maintain coherent learning experiences when students ask cross-disciplinary questions. Generic AI tutors provide one-size-fits-all explanations without understanding the learner's current knowledge state, prior misconceptions, or optimal learning style. When a student asks “Why does gradient descent use calculus, and how would I implement it?”, a single generic tutor cannot coordinate specialized expertise in mathematics, machine learning concepts, and data structures — nor can it explain its reasoning or adapt its teaching based on the student's response. Students are left with fragmented answers and no clear sense of what to learn next, which is especially costly in the high-stakes, time-boxed context of placement preparation.

Problem Analysis / Background

Current tutoring systems, whether generic AI chat assistants or narrow subject bots, suffer from a consistent set of limitations that compound as a learner progresses through a multi-subject curriculum such as DSA, DBMS, Mathematics and AIML:

- Lack of subject specialization — single-agent systems cannot maintain deep, distinct domain knowledge across DSA, DBMS, Mathematics and AIML simultaneously; generic tutors tend to gloss over domain-specific nuance and terminology.
- Weak contextual awareness — systems rarely maintain a rich, shared understanding of a student's knowledge state, prior mistakes, learning pace, or preferred explanation style across sessions and subjects.
- Non-explainable routing — when a system routes a question to an agent or resource, it typically does not explain why, so students and educators cannot audit or trust the decision-making.

- Silent cross-subject gaps — prerequisites across subjects (for example, conditional probability being required for Bayes' theorem) go undetected and uncommunicated to the learner.
- No adaptive teaching loop — most systems teach but do not close the loop by validating learning, identifying misconceptions, or adjusting their explanation strategy in response.
- Isolated progress tracking — mastery levels are tracked per subject in silos, so cross-topic learning patterns and root-cause weaknesses stay invisible to both the student and the system.

Who Is Affected

Students preparing for tech placements are the main users. They need to learn DSA, DBMS, Maths, and AIML in a short time and understand how these subjects are connected. **Mentors and placement coordinators** are also affected because they have limited visibility into students' weaknesses and depend on manual check-ins.

Proposed Solution

EduHive is an explainable, adaptive multi-agent learning orchestrator — a teaching system where a Coordinator Agent intelligently routes student questions to specialized Subject Agents (DSA, DBMS, Maths, AIML), each expert in their domain, while keeping every routing and teaching decision visible and auditable.

- Explainable agent selection — when the Coordinator routes a question, it logs and displays which agent(s) were chosen and why, making routing intelligence transparent to students and educators.
- Shared student context — all agents access a unified Student Profile containing learning history, mastery levels per topic, prior misconceptions, and preferred explanation style, so responses stay personalized and connected to prior learning.
- Cross-subject collaboration — when a question spans multiple subjects, the Coordinator orchestrates simultaneous collaboration (e.g. the Maths Agent explains the calculus, the AIML Agent explains the optimization intuition, the DSA Agent explains the implementation) and synthesizes a single unified answer.
- Adaptive teaching — the system detects confusion via sentiment and interaction analysis, diagnoses misconceptions at the root-cause level, and adapts its teaching style (visual, step-by-step, or analogy-based) accordingly.
- Complete Teach→Test→Diagnose→Adapt cycle — the system explains a concept, generates a knowledge check, evaluates the student's response, identifies learning gaps, updates the student model, and recommends the next best action.
- Intelligent mastery tracking — combines Bayesian Knowledge Tracing (BKT) for interpretability with Deep Knowledge Tracing / LSTM to capture cross-topic learning patterns, enabling early detection of systemic misconceptions.
- Human teacher escalation — when a student repeatedly struggles, the system can recommend escalation to a human teacher and provide detailed context about the student's state.

SECTION 3 — RESEARCH

Research Conducted

Research conducted to inform EduHive's design spans five areas, summarized below:

- Multi-agent learning orchestration — studied how educational AI systems can coordinate multiple specialized agents (subject experts) while maintaining unified learner context, focusing on agentic AI frameworks that let agents reason about uncertainty and collaborate without heavy centralized orchestration overhead.
- Explainability in AI education — reviewed literature on transparent AI decision-making in educational contexts, which emphasizes that students must understand why a system made a teaching choice in order to build trust and metacognitive awareness.
- Knowledge tracing methods — analyzed both classical (Bayesian Knowledge Tracing) and deep learning (Deep Knowledge Tracing with LSTMs) approaches to modeling student knowledge state, and their trade-offs in interpretability, data requirements, and cross-topic pattern detection.
- Adaptive learning design — examined literature on learner-centered adaptive systems, confusion detection, misconception diagnosis, and the teach-test-diagnose-adapt loop as a design pattern.
- Indian tech placement prep context — validated that DSA, DBMS, Maths and AIML are the four core pillars of placement preparation in India, making this specialization both technically sound and market-aligned.

Existing Solutions & Gap Analysis

Existing Approach	Strength	Gap EduHive Addresses
Generic single-agent tutors (Chegg Tutor, ChatGPT-style)	Broad, conversational explanations across all subjects	Lack subject specialization and domain-specific reasoning depth; no explicit learner state or adaptation; no visibility into reasoning or routing.
Subject-specific chatbots	Deep within a single subject domain	Cannot handle cross-subject questions or detect prerequisites; no shared learner profile across subjects.
LMS with knowledge tracing (ALEKS, Carnegie Learning)	Sophisticated knowledge tracing and adaptive recommendation	Requires significant tuning and domain-specific content authoring; treats the learner one subject at a time, not agentially.

References / Sources

- Corbett, A. T., & Anderson, J. R. (1994). Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge. *User Modeling and User-Adapted Interaction*, 4(4), 253–278 — foundational work on Bayesian Knowledge Tracing for learner modeling.
- Piech, C., et al. (2015). Deep Knowledge Tracing. *Advances in Neural Information Processing Systems (NeurIPS)* — deep learning approach to knowledge tracing; informs our LSTM-based mastery model.
- VanLehn, K. (2011). The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems. *Educational Psychology Review*, 23(3), 309–342 — synthesis of tutoring effectiveness research that motivates adaptive multi-agent design.
- Bloom, B. S. (1984). The 2 Sigma Problem: The Search for Methods of Group Instruction as Effective as One-to-One Tutoring. *Educational Researcher*, 13(6), 4–16 — benchmark for one-on-one tutoring; motivates our multi-agent specialist design.
- Technical foundation: FastAPI documentation, PyTorch official tutorials, scikit-learn API reference, and Supabase PostgreSQL documentation.

- Indian tech placement landscape: analysis of interview-prep curricula from LeetCode (DSA), Mode Analytics (SQL/DBMS) and Kaggle (AIML), plus consultation with placement officers at engineering institutions.

SECTION 4 — TECHNICAL DOCUMENTATION

Technology Stack

Layer	Choice	Why
Frontend	React + Vite + Tailwind + Recharts	Fast to build; Recharts renders the Student Brain progress bars with minimal effort.
Backend	FastAPI (Python)	One process serves the API and loads trained models (joblib / .pt) directly — no separate ML microservice under time pressure.
Orchestration	Hand-rolled Coordinator via LLM function/tool-calling	Fastest to build and fully transparent for the Agent Trace panel; skips heavyweight agent frameworks for a 24h build.
Database	Supabase (Postgres)	Source of record for students, conversations, messages, assessments, mistakes, recommendations, agent_routing_log.
Knowledge graph	Adjacency tables in Postgres (student_topic_edges)	Avoids standing up Neo4j — the highest-risk new-infra item for a 24h window — while still answering “what’s this student weak in / what’s a prerequisite for what”.
ML training	scikit-learn + PyTorch (CPU), Colab/Jupyter for the DKT run	Model artifacts are saved and loaded at FastAPI startup.
Auth	Supabase Auth (guest/anonymous login for demo)	Fast, matches the DB choice.
Deployment	Frontend → Vercel; Backend → Render/Railway; DB/Auth → Supabase	Near-zero-config deploys with free tiers; no Docker required.

AI/ML Architecture — Three-Tier Approach

Tier 1 — Classical ML (Subject/Intent Router). Purpose: fast, deterministic, low-cost subject classification before involving the LLM Coordinator. Method: TF-IDF (up to 3000 features, bigrams) + Logistic Regression (scikit-learn), trained on ~150–250 LLM-generated synthetic examples per subject across 5 classes (DSA, DBMS, Maths, AIML, General). Output is a probability distribution across the 5 subjects plus a highest-confidence score. Hybrid inference logic: if confidence ≥ 0.6 , use the router prediction (instant, free, no LLM cost); if confidence < 0.6 , fall back to the LLM Coordinator for semantic classification. Router confidence, predicted subject and fallback status are logged for the Agent Trace. This keeps latency and cost low for well-characterized questions while the fallback provides a safety net for ambiguous edge cases; the 0.6 threshold is configurable based on false-positive/false-negative trade-offs observed during testing.

Tier 2 — Deep Learning (Mastery Modeling). Purpose: capture student knowledge state and cross-topic learning patterns. Bayesian Knowledge Tracing (BKT) models $P(\text{skill mastery} \mid \text{prior interactions})$ via Bayes' rule and is interpretable and cold-start friendly, but treats each skill independently. Deep Knowledge Tracing (DKT) reads a sequence of (skill_id, correct/incorrect) events through an Embedding → LSTM (hidden=64) → linear/sigmoid output, trained on ~800 synthetic student interaction sequences (~30 interactions each); it captures cross-topic learning momentum (e.g. struggling with conditional probability predicts struggling with

Bayes' theorem) but needs historical data, mitigated here with synthetic training for the proof-of-concept. Hybrid strategy: use BKT by default for new students, and switch to DKT as interaction data accumulates and its confidence is high enough — both remain transparent to the student.

Tier 3 — Agentic GenAI (Specialists, Synthesis, Reasoning). Purpose: natural-language explanations, misconception diagnosis, adaptive teaching, and cross-agent synthesis. Components: an LLM Coordinator that routes questions to specialist agents and logs decisions; five Specialist Agents, each expert in one domain, that explain concepts, generate knowledge checks and diagnose misconceptions; a Response Synthesizer that unifies multi-agent responses into a coherent answer (with an optional verifier for consistency); and an Adaptive Teaching Engine that uses confusion detection and mastery state to adjust explanation depth, style, or analogy choice. LLM foundation: the Claude API (Anthropic) via function/tool-calling for deterministic agent coordination. Key reasoning tasks include misconception diagnosis (reasoning about likely root cause — sign error, definition confusion, missing base case, etc.), cross-subject root-cause detection, adaptive teaching style adjustment, cross-agent synthesis, and next-best-action recommendation. Generating natural, adaptive explanations and reasoning about misconceptions is exactly where LLMs excel — training a model from scratch would waste the 24 hours of a hackathon, so this is the right tool for the job.

Confusion Detection (Stretch Goal)

Current baseline: simple keyword matching (e.g. “I don't understand”). Proposed upgrade: a pretrained DistilBERT-SST2 sentiment model as a baseline signal, combined with hand-written features (message length drop, repeated questions, punctuation density), optionally fine-tuned on ~100 labeled “confused” vs. “neutral” examples. The output is a continuous confusion score (0–1) rather than a binary hit, enabling more nuanced adaptation triggers.

Technical Flow / System Architecture Diagram

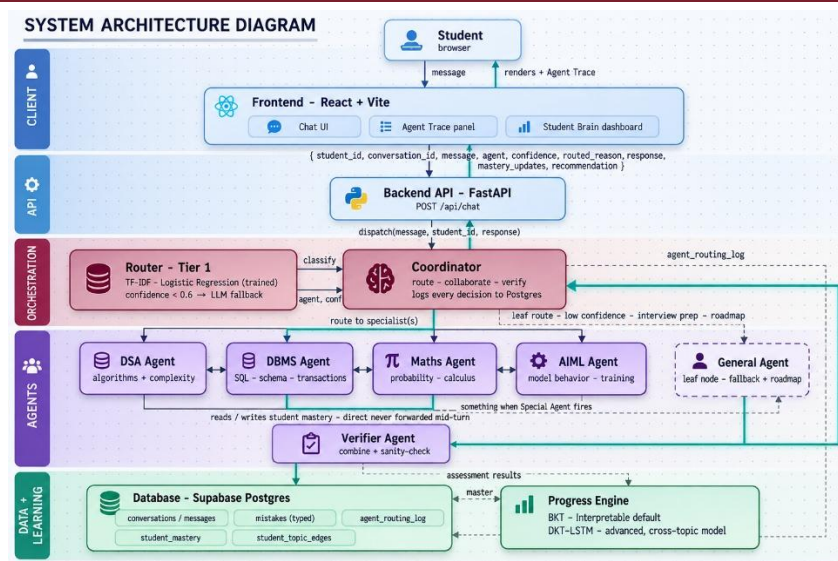


Figure 1: End-to-end request flow — from student question through routing, coordinator-led specialist collaboration, synthesis, the knowledge-check/diagnosis loop, and mastery update, with optional human-teacher escalation.

Step-by-Step Flow

- Student input — the student submits a question via the chat interface (React frontend).
- Router classification — the question is passed to the trained TF-IDF + Logistic Regression router, which predicts the subject/intent (DSA, DBMS, Maths, AIML, or General) with a confidence score.
- Confidence check — if confidence ≥ 0.6 , the predicted subject is trusted; if < 0.6 , the question is handed to the LLM Coordinator for semantic classification (fallback).

- Student context retrieval — the Coordinator fetches the student's profile from PostgreSQL, including prior interaction history, mastery scores per topic (from BKT/DKT), prior misconceptions, learning-style preference, and cross-subject prerequisite status.
- Specialist agent selection — based on subject classification and student context, the Coordinator selects one or more specialist agents: DSA, DBMS, Maths, AIML, or General (fallback; also handles career/interview prep).
- Multi-agent collaboration — if the question spans multiple subjects, the Coordinator orchestrates parallel or sequential calls to multiple specialists, each receiving the original question, relevant context from the student profile, and cross-agent decision logs to prevent redundant explanations.
- Response generation — each specialist agent generates an explanation tailored to the student's current mastery level, prior misconceptions, and preferred learning style.
- Synthesis & validation — if multiple agents responded, the Coordinator synthesizes their answers into a coherent, unified response; an optional verifier agent sanity-checks the combined response for consistency.
- Knowledge check generation — the system generates a targeted knowledge-check question to verify the student's understanding.
- Student response evaluation — the student's answer to the knowledge check is scored (correct/incorrect) and analyzed for misconceptions.
- Misconception diagnosis — if the student answered incorrectly, the system diagnoses the likely root cause (e.g. sign error, definition confusion, off-by-one) and logs it with a taxonomy tag.
- Mastery state update — the student's mastery score for the topic is updated using BKT (interpretable, cold-start-friendly) or DKT with LSTM (captures cross-topic learning patterns as data accumulates).
- Teaching strategy adaptation — the Coordinator adjusts its approach based on confusion detection (sentiment + interaction patterns), repeated mistakes, and engagement level (response latency, question length).
- Progress & recommendation — the system updates the student's progress dashboard and generates a personalized next-best-action recommendation (e.g. “practice conditional probability before Bayes' theorem”).
- Optional escalation — if the student has struggled with a concept across multiple interactions, the system recommends escalation to a human teacher and provides a full diagnostic report.

Database schema (key tables): students (user profiles, preferences, learning goals); interactions (every Q&A, timestamp, agent involved, correctness); mastery_scores (per-student per-skill, BKT/DKT estimates, last_updated); mistakes (conceptual errors, taxonomy such as sign_error / definition_confusion, root-cause linked); agent_logs (routing decisions, confidence scores, agent trace); prerequisites (cross-subject dependency graph — which skills must precede others).

Working / Implementation Approach

Tech integration. Frontend (React) sends the question via Axios POST to the /api/ask endpoint. Backend (FastAPI) loads the router model (joblib) and DKT model (PyTorch) at startup, loads the LLM client (Claude API), queries PostgreSQL for student context, orchestrates specialist agents via function calling, updates the database with results, and returns the response plus a full trace to the frontend. All state is persisted to PostgreSQL via the Supabase client.

Error handling & fallbacks. Router confidence < 0.6 triggers LLM fallback classification; an LLM timeout falls back to the router prediction plus the General agent; missing student context falls back to default preferences; a specialist-agent error switches to the General agent with an apology; and a database outage triggers in-memory response caching for graceful degradation.

Model serving. The router model (.joblib) is loaded at backend startup with inference around 5–10ms per question; the DKT model (.pt) is loaded at startup with inference around 50–100ms per student interaction; LLM calls go to the Anthropic Claude API with typical latency of 2–5 seconds.

Monitoring & logging. The agent_logs table records every routing decision, confidence score, and agent chosen; the mistakes table tags student errors with a misconception taxonomy; the mastery_scores table tracks per-student per-skill evolution over time; and the interactions table maintains a full audit trail of Q&A and correctness.

Scaling considerations. For ~1,000 concurrent students, scale backend replicas on Render/Railway; for 100,000+ interactions/day, add a Redis cache for frequent questions; for large DKT models, deploy as a separate microservice; and for high LLM load, implement request queuing and rate limiting.

GitHub Repository

Repository: <https://github.com/shendeyugandhar5-ai/hackathon-starter/tree/main>

The repository contains /frontend (React + Vite + Tailwind CSS source), /backend (FastAPI Python source with agent orchestration), /models (trained router — TF-IDF + Logistic Regression — and DKT/LSTM artifacts), /database (PostgreSQL schema and initialization scripts), /docs (architecture diagrams, API documentation, model training notebooks), a comprehensive README with setup and deployment guide, and a .env.example template for API keys and database URLs.

Expected Impact / Outcomes

- Students get a single conversational entry point that still delivers subject-expert-level depth across DSA, DBMS, Maths and AIML.
- Transparent routing and agent traces let students and mentors audit why a particular explanation path was chosen, building trust in the system.
- Cross-subject prerequisite detection surfaces root-cause weaknesses earlier than siloed, per-subject tracking would.
- A closed teach-test-diagnose-adapt loop turns passive reading into active, verified learning with measurable mastery gains over time.

Future Scope / Roadmap

- Expand specialist coverage beyond DSA/DBMS/Maths/AIML to Operating Systems, Computer Networks, and System Design.
- Replace synthetic DKT training data with real, anonymized interaction logs once the platform has live usage.
- Add a fine-tuned confusion-detection classifier in place of the DistilBERT-SST2 baseline for sharper adaptation triggers.
- Introduce a mentor/teacher dashboard that surfaces the Agent Trace and diagnostic reports for an entire cohort, not just one student.
- Support voice input and richer visual explanations (auto-generated diagrams) for visual learners identified by the Adaptive Teaching Engine.

SECTION 5 — ORIGINALITY & PLAGIARISM DECLARATION

☒ We confirm that the solution submitted by our team is developed during the hackathon. We have not submitted a pre-built project as our primary solution. Any open-source code, APIs, datasets, libraries, templates, or AI-assisted development used by us have been appropriately acknowledged below.

Open-Source / External Resources Used

Category	Resource
Frontend	React + Vite + Tailwind + Recharts
Backend	FastAPI (Python)

Category	Resource
Orchestration	Hand-rolled Coordinator via LLM function/tool-calling
Database	Supabase (Postgres)
Knowledge graph	Adjacency tables in Postgres (student_topic_edges)
ML training	scikit-learn + PyTorch (CPU), Colab/Jupyter for the DKT run
Auth	Supabase Auth (guest/anonymous login for demo)
Deployment	Frontend → Vercel; Backend → Render/Railway; DB/Auth → Supabase

AI Tools Used

Claude AI, ChatGPT, and Gemini AI were used during development — for architecture and design discussion, code generation and debugging assistance, and drafting/refining project documentation. All AI-assisted code and text was reviewed and adapted by the team before inclusion in the submission.

SECTION 6 — UPLOADS / ATTACHMENTS CHECKLIST

- Technical Flow / Architecture Diagram — included above (Figure 1).
- Research / Reference Document (if applicable)
- Project Documentation / Technical Report — this document
- GitHub Repository Link — included in Section 4
- Screenshots of Working Prototype — to be attached separately
- Dataset / Sample Data (if applicable) — synthetic training data described in Section 4, Kaggle Datasets
- Additional Diagrams (if any)

Team Leader Signature: **Soham Parab**

Date: **11-09-2026**